Best practices for data persistence in Docker

In this train, we'll apply best practices to keep data persistent, secure, and accessible across container restarts and deployments.

Learning objective

- Be able to apply best practices to keep data persistent, secure, and accessible across container restarts and deployments.

Best Practices for Data Persistence

> By default, Docker containers are ephemeral—any data written inside a container is lost when the container stops or is removed. To persist data, we use:
>
> - Volumes (Managed by Docker)
> - Bind Mounts (Mapped to host filesystem)
> - tmpfs Mounts (In-memory storage, not persistent)

1. Use Named Volumes for Persistent Data

Named volumes are managed by Docker and are the best choice for persistent storage.
Example: Creating and Using a Named Volume
Creating: `docker volume create mydb_data`
Usage: `docker run -d --name mysql_db -v mydb_data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=secret mysql:latest`

- **Persists** even if the container is removed.
- **Secure**: Only accessible by Docker-managed processes.

2. Use Bind Mounts for Development

Bind mounts link a host directory to a container directory. Useful for development but less secure in production.

Example: Bind Mount for Development
`docker run -d --name dev_app -v $(pwd)/app:/app node:latest`

- **Pros:** Easy file sharing between host and container.
- **Cons:** Host dependency, potential security risks.

3. Backup & Restore Volumes

Regularly back up Docker volumes to avoid data loss.
Example: Backup a Volume
`docker run --rm -v mydb_data:/source -v $(pwd):/backup alpine tar cvf /backup/mydb_backup.tar /source`

Example: Restore a Volume
`docker run --rm -v mydb_data:/target -v $(pwd):/backup alpine tar xvf /backup/mydb_backup.tar -C /target --strip 1`

4. Using Docker Compose for Persistent Storage

Define volumes in `docker-compose.yml` for better maintainability. Example: `docker-compose.yml` with Volumes

```
version: '3.8'

services:

  db:

        image: postgres:latest

        volumes:
      - pg_data:/var/lib/postgresql/data

        environment:
      POSTGRES_PASSWORD: secret

volumes:

      pg_data:
```

- Run with: docker compose up -d

5. Avoid Storing Sensitive Data in Volumes

Use Docker Secrets or environment variables for credentials.
Encrypt volumes if storing sensitive data. Example: Using Docker Secrets (Swarm Mode)

```
echo "my_db_password" | docker secret create db_password - docker service create --name mysql --secret db_password mysql:latest
```

Conclusion

By following Docker data persistence best practices—such as using named volumes for production, bind mounts for development, read-only volumes for security, and regular backups—you can ensure your containerised applications retain data reliably across restarts and deployments. Leveraging tools like Docker Compose and Docker Secrets further enhances maintainability and security, preventing data loss and unauthorised access.

Key Takeaways

- **Named volumes** are best for production persistence.
- **Bind mounts** are useful for development but avoid in production.
- **Read-only volumes** improve security.
- **Backup volumes** regularly to prevent data loss.
- **Docker Compose** simplifies volume management.